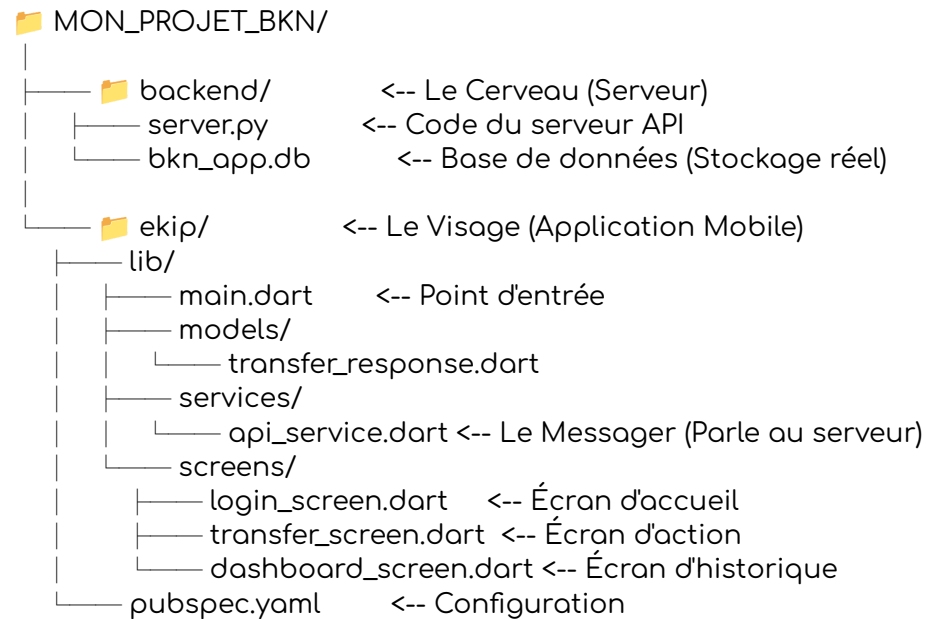


Récapitulatif complet de tout du travail pour créer cette application de transfert d'argent BKN

Pour transformé un simple concept en une application multi-utilisateurs fonctionnelle, avec un serveur réel, une base de données et un système d'historique

Structure du Projet Finale

Voici les dossiers clés que j'ai mis en place :



1.Détail des Fichiers Modifiés ou Créés

Fichier	Statut	Rôle & Modifications
server.py	Créé	C'est le cœur du système. - Il crée automatiquement la base de données SQLite. - Il gère 3 API : <code>GET /balance</code> (Solde), <code>POST /transfer</code> (Virement), <code>GET /history</code> (Historique). - Il contient la logique métier (qui envoie à qui, vérification du solde).
bkn_app.db	Généré	Fichier binaire créé automatiquement par Python. Il stocke de façon permanente les utilisateurs, leurs soldes et toutes les transactions passées.

2. Frontend (Application Flutter)

Fichier	Statut	Rôle & Modifications
pubspec.yaml	Modifié	Ajout de la librairie <code>http</code> pour permettre à l'application de se connecter à Internet/au serveur.
main.dart	Modifié	Point de départ de l'app. Nous avons retiré le bandeau "DEBUG" et défini <code>LoginScreen</code> comme la première page à afficher.
api_service.dart	Modifié	Le pont entre Flutter et Python. - Avant : Faisait semblant (simulation). - Maintenant : Envoie de vraies requêtes HTTP vers <code>10.0.2.2:8000</code>. Gère les méthodes <code>getSolde</code>, <code>transfererMontant</code>, et <code>getTransactions</code>.
login_screen.dart	Créé	Nouvelle page d'accueil moderne. - Permet de choisir son identité (User 1 ou User 2). - Permet d'accéder au Dashboard sans se connecter. - Design soigné (dégradé bleu, cartes ombrées).

<code>transfer_screen.dart</code>	Modifié	La page principale de transfert. - Affiche le solde réel depuis la DB. - Permet de saisir un montant et d'envoyer l'argent. - Affiche le succès ou l'échec retourné par le serveur.
<code>dashboard_screen.dart</code>	Créé	Page de "Contrôle". - Affiche l'historique complet des échanges. - Onglet 1 : Liste visuelle (User 1 → User 2). - Onglet 2 : Vue JSON brute pour les développeurs.

Flux de Fonctionnement (Comment tout se connecte)

1. Démarrage : Le serveur Python (`server.py`) démarre et ouvre la base de données (`bkn_app.db`).
2. Connexion : L'utilisateur lance l'app, arrive sur `LoginScreen`. Il choisit "User 1".
3. Affichage : `TransferScreen` demande à `ApiService` : *"Quel est le solde de User 1 ?"*.
4. Requête : `ApiService` interroge le serveur Python. Le serveur lit la DB et répond *"5000 €"*.
5. Action : L'utilisateur envoie 50€. `ApiService` dit au serveur : *"User 1 envoie 50€"*.
6. Traitement : Le serveur Python :
 - Vérifie les fonds.
 - Soustrait 50 à User 1.
 - Ajoute 50 à User 2.
 - Enregistre la transaction dans la table historique.
 - Confirme le succès.
7. Vérification : On peut aller sur le `DashboardScreen` pour voir la trace indélébile de cette opération.